CSA0609-DESIGN AND ANALYSIS OF ALGORITHMS

KRISHA MEENATCHI M (192511338)

Exp-13: Merge Sort with Comparison Count

Aim:

To sort an array using Merge Sort and count the number of comparisons performed during sorting.

Algorithm:

1. Read the array elements.
2. Divide the array into two halves recursively.
3. Sort each half using Merge Sort.
4. Merge the two sorted halves.
5. Count each comparison made while merging.
6. Display the sorted array and total comparison count.

Source Code:

```
comparisons = 0
def merge_sort(arr):
    global comparisons
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    result = []
    i = 0
    j = 0
    while i < len(left) and j < len(right):
```

```
        comparisons += 1
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
n = int(input("Enter number of elements: "))
arr = list(map(int, input("Enter array elements: ").split()))
arr = merge_sort(arr)
print("Sorted Array:", arr)
print("Number of Comparisons:", comparisons)
print("Time Complexity: O(n log n)")
print("Space Complexity: O(n)")
```

Output:

```
Enter number of elements: 6
Enter array elements: 38 27 43 3 9 82
Sorted Array: [3, 9, 27, 38, 43, 82]
Number of Comparisons: 10
Time Complexity: O(n log n)
Space Complexity: O(n)
```

Result:

The array was successfully sorted using Merge Sort, and the number of comparisons was counted.